allreduce/recursive_doubling

An AgentMPI collective scaling analysis

Abstract

Recursive doubling is the latency-optimal allreduce: every rank exchanges its accumulator with a partner at distance 2^k and folds locally, so after $\lceil \log_2 p \rceil$ rounds every rank holds the full fold without any rank having been a root. This family measures it in 21 runs at 13 distinct process counts from 2 to 32, and it contains the one genuine defect the collective sweep exposes. At every non-power-of-two size the collective under-reported its own traffic by exactly $\text{rem} = p - 2^{\lfloor \log_2 p \rfloor}$ messages — 4 reported against 5 logged at $p = 3$, 28 against 30 at $p = 10$, 80 against 88 at $p = 24$ — and at 2, 4, 8, 16 and 32 the two figures agree exactly. The messages themselves were sent, delivered and folded correctly: the result recursive doubling delivers is the same content-addressed object that `reduce_bcast` delivers at all eight non-power-of-two sizes where both algorithms ran. Only the *count* was wrong, which is the more dangerous direction, because the cost report, the calibration and every table derived from a run read the reported count rather than the log, and an under-reporting collective looks cheaper than it is. The cause is one missing `tr.sent()` call in the remainder pre-phase, where both partners exchange via `sendrecv` but only the odd-numbered rank recorded its half; the trace localises the shortfall to exactly one message on each of ranks $0, 2, \ldots, 2\,\text{rem} - 2$. The fix is committed (`62960b7`) and postdates these traces, so this archive preserves the defect rather than hiding it. The single thing to take away is why it needed a family to find: at any one p the gap is a two-message discrepancy that could be a lost event or an off-by-one anywhere, and it is only against 9 rows that agree and 12 that fall short by precisely the remainder that the discrepancy stops being noise and names the code path that caused it.

1 What this family is

The `allreduce` collective under the `recursive_doubling` algorithm, measured at 21 process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- **[ok]** All 21 measured sizes logged exactly the number of messages the closed form predicts.
- **[warning]** At 12 size(s) the collective's self-reported count disagrees with the traffic the fabric logged, and every one of them is a non-power-of-two size, which points at the remainder path: $p = 3$ (reported 4, logged 5), $p = 3$ (reported 4, logged 5), $p = 5$ (reported 10, logged 11), $p = 5$ (reported 10, logged 11), $p = 6$ (reported 12, logged 14), $p = 7$ (reported 14, logged 17), $p = 7$ (reported 14, logged 17), $p = 10$ (reported 28, logged 30)

- [note] Wall times span 0.014–0.252 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

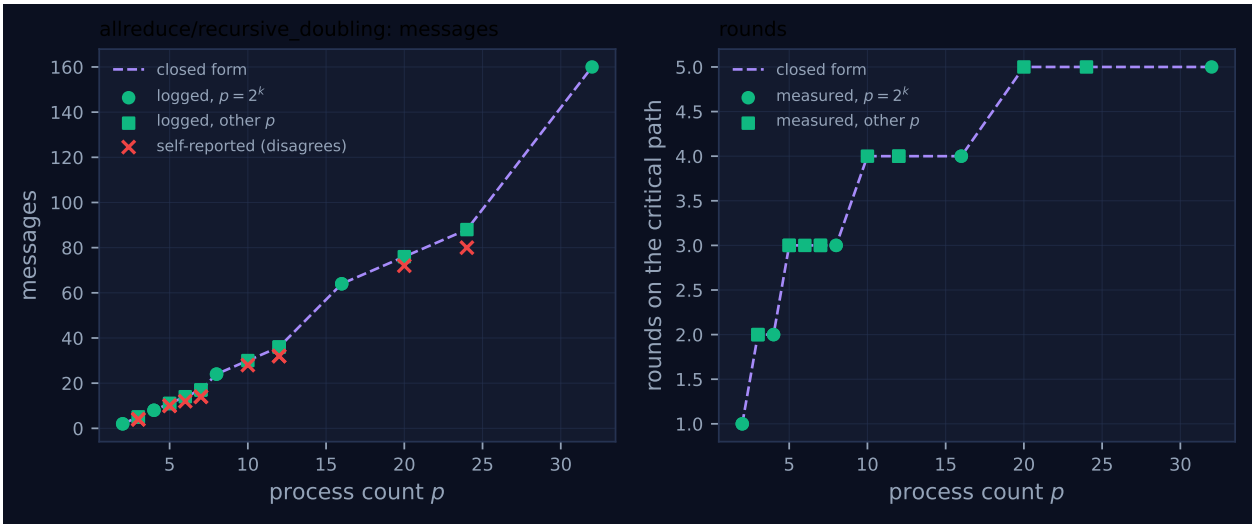

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	1	1	0.014	✓	✓
2	`mb-smoke`	2	2	2	1	1	0.015	✓	✓
3	`mb-free`	5	5	4	2	2	0.016	✓	×
3	`mb-smoke`	5	5	4	2	2	0.030	✓	×
4	`mb-free`	8	8	8	2	2	0.018	✓	✓
4	`mb-smoke`	8	8	8	2	2	0.027	✓	✓
5	`mb-free`	11	11	10	3	3	0.027	✓	×
5	`mb-smoke`	11	11	10	3	3	0.028	✓	×
6	`mb-free`	14	14	12	3	3	0.143	✓	×
7	`mb-free`	17	17	14	3	3	0.049	✓	×
7	`mb-smoke`	17	17	14	3	3	0.035	✓	×
8	`mb-free`	24	24	24	3	3	0.047	✓	✓
8	`mb-smoke`	24	24	24	3	3	0.049	✓	✓
10	`mb-free`	30	30	28	4	4	0.071	✓	×
12	`mb-free`	36	36	32	4	4	0.085	✓	×
12	`mb-smoke`	36	36	32	4	4	0.069	✓	×
16	`mb-free`	64	64	64	4	4	0.130	✓	✓
16	`mb-smoke`	64	64	64	4	4	0.080	✓	✓
20	`mb-free`	76	76	72	5	5	0.130	✓	×
24	`mb-free`	88	88	80	5	5	0.153	✓	×
32	`mb-free`	160	160	160	5	5	0.252	✓	✓

4 How the algorithm scales

4.1 Where the cost comes from

The power-of-two case is the whole algorithm. In round k rank r exchanges accumulators with $r \oplus 2^k$ and folds what arrives, so after round k every rank's accumulator covers the 2^{k+1}-rank subcube containing it; after $\log_2 p$ rounds it covers all of them. Every rank sends in every round, which gives $p \log_2 p$ messages and $\log_2 p$ rounds — exactly p times the message count of a binomial `reduce`, in exchange for producing the answer everywhere instead of at one place.

Non-power-of-two p breaks the hypercube, and `algorithms.allreduce` applies the standard correction. Let $\text{pof2} = 2^{\lfloor \log_2 p \rfloor}$ and $\text{rem} = p - \text{pof2}$. The lowest $2\,\text{rem}$ ranks pair up: each even rank exchanges with its odd neighbour and folds, each odd rank hands over its contribution and sits out. What remains is a set of exactly pof2 participants, renumbered so that the hypercube exchange works, and at the end each sitting-out rank is sent the answer. Three phases, and the closed form in `cost.FORMULAS` is their sum:

$$\text{messages} = \underbrace{2\,\text{rem}}_{\text{pre}} + \underbrace{\text{pof2} \cdot \log_2 \text{pof2}}_{\text{doubling}} + \underbrace{\text{rem}}_{\text{post}} = 3\,\text{rem} + \text{pof2} \log_2 \text{pof2}.$$

The pre-phase is a `sendrecv`, so it is *two* messages per pair, not one; the post-phase is a one-way `_csend`, so it is one. That asymmetry is where the defect lives, and it is worth holding on to.

The decomposition is not a derivation I am asking the reader to trust: it is directly visible in the internal message tags. Grouping the 30 `msg.send` events of `mb-free__coll-allreduce-recursive_doubling-10` by the last component of their tag gives `pre` 4, `0` 8, `1` 8, `2` 8, `post` 2 — that is $2\,\text{rem} = 4$, three doubling rounds of $\text{pof2} = 8$, and $\text{rem} = 2$, summing to the predicted 30. The same grouping at $p = 24$ gives `pre` 16, four rounds of 16, and `post` 8, summing to 88. Every size in the table decomposes this way.

4.2 What the figure shows

The messages panel of Figure 1 is not a smooth curve, and the reason is the $\text{pof2} \log_2 \text{pof2}$ term. Within a power-of-two bracket the doubling term is frozen and only the $3\,\text{rem}$ term moves, so cost grows at three messages per added rank: $p = 16$ costs 64, $p = 20$ costs 76, $p = 24$ costs 88. Crossing a power of two the doubling term jumps, and it jumps hard — from $p = 24$'s 88 to $p = 32$'s 160, an 82% increase for a 33% increase in population. The visible consequence is a family of shallow segments punctuated by cliffs at 16 and 32, and it has a practical reading: a harness at $p = 24$ that adds eight ranks pays 72 extra messages, whereas one at $p = 16$ that adds eight pays 24. Below a power of two, population growth is cheap; crossing one is not.

The rounds panel steps where $\lceil \log_2 p \rceil$ steps, at 3, 5, 9 and 17, and the measured points sit on the line at all 21 rows. Both marker classes lie on it, which is the first thing worth confirming and the least interesting.

The interesting thing is the red crosses. Twelve of them, at $p = 3, 5, 6, 7, 10, 12, 20$ and 24 (with $3, 5, 7$ and 12 measured twice, once per campaign), each sitting below the closed-form line by a visible amount that grows with p. The green logged counts sit *on* the line at those same sizes. Two numbers that should be the same number, plotted against each other, disagreeing in a pattern.

5 Powers of two and everything else

5.1 The defect, stated precisely

At every one of the 12 non-power-of-two runs, the count the collective reported in its `coll.allreduce` events is short of the number of `msg.send` events the fabric holds, and the shortfall is exactly rem:

p	pof2	rem	logged	self-reported	shortfall
3	2	1	5	4	1
5	4	1	11	10	1
6	4	2	14	12	2
7	4	3	17	14	3
10	8	2	30	28	2
12	8	4	36	32	4
20	16	4	76	72	4
24	16	8	88	80	8

and at $p = 2, 4, 8, 16, 32$ the shortfall is zero. Both campaigns agree at every size measured twice, so this is not a scheduling artefact of one execution.

Attributing the missing sends to ranks settles the cause. Differencing each rank's `msg.send` count against the `messages_sent` field of its own `coll.allreduce` event, the gap at $p = 24$ is exactly one message on each of ranks 0, 2, 4, 6, 8, 10, 12, 14 — the rem = 8 even members of the pre-phase pairing — and zero everywhere else. Each of those ranks made exactly one `:pre`-tagged send. At $p = 10$ it is ranks 0 and 2; at $p = 7$, ranks 0, 2 and 4; at $p = 3$, rank 0 alone. In every case the set of ranks with a gap is $\{0, 2, \ldots, 2\,\mathrm{rem} - 2\}$ and the gap is one.

The code says the same thing. Both branches of the pre-phase call `comm.sendrecv`, which sends one message and receives one; the odd branch follows it with `tr.sent(...)` and, in the traces, the even branch did not. Two messages crossed the fabric, one was counted. This is the whole defect: rem pairs, one uncounted send each, rem missing messages, at exactly the sizes where rem > 0.

5.2 Why the messages were fine and the count was not

It matters that this is an instrumentation defect and not a communication one, and the archive can distinguish the two rather than taking the claim on trust. If the pre-phase send had been lost rather than merely unrecorded, the even rank's fold would have been missing a contribution and the result would have been wrong. It was not. Comparing the content-addressed digest of the answer recursive doubling distributes in its post-phase against the digest `reduce_bcast` broadcasts at the same p, the two match at all eight non-power-of-two sizes where both algorithms were run: `4e07408562be` at $p = 3$, `4a44dc153642` at $p = 10$, `c2356069e9d1` at $p = 24$, and so on. Two structurally different algorithms, folding in different orders, produced byte-identical results. The remainder path computes correctly.

So the defect is confined to the number the collective reports about itself, and that is the worse place for it to be. `cost.summarise` builds its per-collective `messages` bucket from the `messages_sent` payload field, not from the `msg.send` stream; so does every table a harness prints from a run. An allreduce at $p = 24$ that actually sent 88 messages presented itself as having sent 80, a 9% discount, and the discount is systematic rather than noisy — it is present in every non-power-of-two run and absent in every power-of-two one. A cost model may be wrong in either direction, but a cost

model that is wrong in the direction of cheapness will be trusted, and the algorithm-selection logic in `cost.best_algorithm` is exactly the consumer that would be misled.

5.3 Why one run could not have shown this

The per-run analysis of `mb-free__coll-allreduce-recursive_doubling-10` does flag a discrepancy: its findings list says the collective "reports 28 messages but the fabric logged 30". The viewer screenshot for that run shows the same thing more starkly, and without comment — the stats row's MESSAGES tile reads **30**, taken from the fabric, and the COLLECTIVES panel a few centimetres below reads MESSAGES **28**, taken from the collective's self-report. Two numbers on one screen, disagreeing by two, neither marked.

What that single run cannot tell you is what kind of thing it is. A two-message gap at $p = 10$ is consistent with a dropped event, a race in the emit path, a rank that finalised before its last send was recorded, an off-by-one in the tag attribution, or a miscount in any of the algorithm's three phases. Nothing in the run distinguishes them, and the honest per-run conclusion is "something does not add up here".

The family view converts that into an identification, in three steps that each need more than one point. First, the gap is zero at 2, 4, 8, 16 and 32 and non-zero at every other size, which excludes every explanation that does not depend on p's factorisation and points at the remainder path specifically. Second, the gap equals rem across a range in which rem takes the values 1, 2, 3, 4 and 8 — a coincidence at one size, a functional relationship across eight. Third, the doubling and post phases are common to sizes with and without a gap while the pre-phase is not, so within the remainder path the pre-phase is the only candidate. A constant shortfall equal to the remainder is a pattern, and a pattern needs more than one point to be one.

The same reasoning explains how the defect survived the test suite. `test_message_counts_match_cost_formulas` ran at $p = 4$ and $p = 8$, where rem $= 0$ and the pre-phase never executes, and it compared the self-report against the closed form — two numbers that can agree with each other while both diverge from the traffic actually sent. Commit `62960b7` adds the missing `tr.sent()` and a test that runs 11 op/algorithm pairs at $p \in \{3, 5, 6, 7, 10\}$ and requires three numbers to agree, including the one nothing had checked: the count of `msg.send` events the fabric holds. That commit postdates these traces, so the runs archived here still exhibit the defect. That is a property of a committed archive working as intended: the traces are the record of what the code did when it ran, and a record that silently updated itself to match a later fix would be worth nothing.

5.4 A second, smaller discrepancy in the same formula

The round count deserves the same treatment, because the family view shows it is subject to the same failure mode — self-report and closed form agreeing while both diverge from the log.

The implementation sets `rounds = _ilog2_ceil(p)` and the closed form's first component is `_log2c(p)`; they agree at all 21 rows, and the table records ✓ accordingly. But the remainder path has three sequential stages, and the number of dependent communication steps on the longest path is $\log_2 \mathrm{pof2} + 2$, which for non-power-of-two p is $\lceil \log_2 p \rceil + 1$. Counting the distinct tag phases each rank participates in gives exactly that: 5 at $p = 10$, 12 and 20; 6 at $p = 20$ and 24 — against a reported 4, 4, 4, 5 and 5. At every power of two the two figures coincide (4 phases at $p = 16$, 5 at $p = 32$), and at every non-power-of-two size the report is one short.

This is a much smaller matter than the message undercount — one round out of four or five, and in the optimistic direction again — and unlike the message count it is a modelling choice as much as a bug: $\lceil \log_2 p \rceil$ is the figure the textbook quotes. But the comment above the formula in `cost.py` explicitly corrected the *message* count for the non-power-of-two case ("the naive $p \log_2 p$ figure is wrong by up to 25% at non-power-of-two sizes") and left the round count uncorrected on the line above, which suggests oversight rather than intent. I flag it as a discrepancy to check rather than as an established defect, because it is derivable from the code and the tags but has no independent measurement behind it: nothing in these agent-free runs pays a per-round latency large enough to expose it.

6 When to choose this algorithm

6.1 Against `reduce_bcast`, which is the real question

AgentMPI implements two allreduce algorithms and defaults to the other one. The family measurements make the comparison concrete; the two columns below are the logged counts from the two families at matching p and campaign.

	messages		rounds		wire tokens		
p	rec. dbl.	red. + bc.	rec. dbl.	red. + bc.	rec. dbl.	red. + bc.	operator applications
8	24	14	3	6	384	147	24 vs 7
10	30	18	4	8	450	189	28 vs 9
16	64	30	4	8	1024	315	64 vs 15
32	160	62	5	10	2560	651	160 vs 31

Recursive doubling halves the round count and pays 2.6× the messages, 3.9× the wire volume and 5.2× the operator applications for it. The application count is the one that matters for an agent harness and the one no table in the generated artefacts reports: in `reduce_bcast` the operator is applied once per edge of a binomial tree, $p - 1$ times in total; in recursive doubling every rank folds in every round, so the total is $\text{pof2} \log_2 \text{pof2} + \text{rem}$, which is 160 at $p = 32$ against 31. With a semantic operator each application is an agent invocation.

The halved round count buys much less than it appears to, and this is the part of the trade-off that the AgentMPI setting changes. `reduce_bcast`'s $2\lceil \log_2 p \rceil$ rounds are $\lceil \log_2 p \rceil$ reduce rounds, each one an operator application, plus $\lceil \log_2 p \rceil$ broadcast rounds that invoke nothing — the broadcast forwards a content-addressed handle, so an interior rank relays without reading. Under the repository's own default parameters (`CostParams`: $\alpha = 12\,\text{s}$, $\beta^{-1} = 45$ tokens/s, `fabric_s` $= 3\,\text{ms}$) a fold round costs about 38.7 s at a 1200-token payload and a forward round costs about 0.153 s. `cost.predict` therefore puts the two algorithms within half a second of each other at every size: 116.9 s against 116.5 s at $p = 8$, 194.9 s against 194.1 s at $p = 32$. Recursive doubling wins on latency by 0.4%, and `best_algorithm("allreduce", p, n, objective="time")` duly returns it — on a margin of four parts in a thousand, while the same model prices it 70% higher.

Fold depth, the third axis, does not separate them: both are $\lceil \log_2 p \rceil$, and `cost.predict` gives both the same fidelity estimate ($F = 0.7738$ at $p = 32$ for $\delta = 0.05$). What separates them is not depth but *agreement*, and that is the subject of the next subsection.

The recommendation follows: use `reduce_bcast`, which is what the code does. Recursive doubling is the right choice only where the root of a `reduce_bcast` would be a genuine bottleneck that the

model does not capture — a rank whose context budget cannot hold the whole fold, or a harness in which the root's agent is materially slower than the others — and this sweep contains no evidence that either condition arises.

6.2 The divergence question, and what this archive cannot answer

The docstring for the algorithm calls it "a *trap* for lossy operators": each rank performs its own fold sequence, so the p results may differ, and the population silently disagrees about the glossary it just agreed on. The claim needs qualifying in two directions and I want to be careful about both, because the archive supports neither the claim nor its denial.

First, the implementation is more careful than the docstring suggests. Each pairwise combine is ordered canonically: `if comm.rank < partner or op.commutative` puts the lower-ranked operand on the left, otherwise it swaps. Because the renumbering that produces the pof2 participant set is monotone in rank, and because the hypercube exchange gives every rank the same subcube partition at every level, the *expression* each rank evaluates is the same balanced binary fold over ranks in order. Order is therefore not a source of divergence here. What remains is that the operator is *executed* independently $\text{pof2}\log_2\text{pof2} + \text{rem}$ times rather than $p-1$ times, and a language-model operator is not a function: identical operands do not guarantee identical output. Divergence in recursive doubling is a nondeterminism argument, not an associativity argument. `reduce_bcast` is immune for a structural reason rather than a statistical one — it applies the operator $p-1$ times and distributes one object. At $p = 32$ all 31 of its broadcast sends carry the same digest, `e29c9c180c62`; at $p = 10$, all 9 carry `4a44dc153642`. One value, forwarded, byte-identical by construction.

Second, and more importantly: **divergence is unmeasured in this repository.** Across all 496 `coll.*` records that carry the field, `divergence_risk` is `false` every single time. The flag is set to `op.lossy`, and the only operator ever run under recursive doubling in the archive is `SUM` (206 records) and `UNION` (8), both declared `Associativity.EXACT` and therefore not lossy. The one semantic operator in the traces, `GLOSSARY_MERGE`, appears in 8 records and every one of them is under `reduce_bcast`. The semantic-operator-with-recursive-doubling cell of the design is empty. Everything above about fidelity and agreement is derived from the code and the cost model; no measurement in this repository bears on it, and a reader should treat the claim that recursive doubling diverges as a hypothesis the harness is built to test and has not yet tested.

7 Threats to this reading

There are no agents in any of these runs, and nothing here bears on agent behaviour. `metrics.json` for every member records `executors: {none: p}`, zero agent calls, zero tokens in and out, $0.0000. The concurrency block reports `total_busy_s: 0`, `achieved_parallelism: 0.0` and `idle_fraction: 1.0` at every size; those are properties of `bench_collectives`, whose `rank_main` calls `comm.allreduce(1, SUM)` and returns, so a rank has nothing to be busy with. Read them as "not applicable". Every statement above about α, about operator applications and about fidelity comes from `cost.py`'s parameters and formulas, not from these measurements. What the sweep measures is structure: how many messages, in how many phases, to whom.

The wall-clock column is a measurement of a polling schedule, not of the protocol. Wall times span 0.014 s to 0.252 s and rise with p, which invites reading them as a latency curve. They are not one. A blocked receive in `comm.py` sleeps `POLL_INTERVAL` = 10 ms and multiplies the

interval by 1.4 up to `MAX_POLL_INTERVAL` = 250 ms, so a rank whose partner is late discovers the message up to a quarter of a second after it arrived. Any wall-time comparison between algorithms in this sweep is confounded by how deep each algorithm's critical path is in poll intervals, and the effect is large: the `scan/chain` family, whose critical path is $p - 1$ deep, reaches 2.234 s at $p = 32$ with only 31 messages. The message and round counts are exact; the seconds are not a property of AgentMPI's design.

The token volumes in the collectives' self-report and in the log measure different things, and neither is the quantity the cost model wants. At $p = 32$ the fabric logged 2560 tokens across 160 sends, a uniform 16 tokens each, while the collective reported 160 — one per message. The tracker counts `_tok(comm, acc)`, the accumulator alone, which for this benchmark's payload is the integer 1; the fabric counts the serialised envelope `{"acc", "depth", "weight"}`. The 15-token difference is fixed framing overhead, so it is a factor of 16 here and negligible for a realistic payload, but it means the closed form's volume term is untested by this sweep: with a one-token contribution, essentially all measured volume is framing. The undercount established above is about message *counts*, which are unambiguous, and I have deliberately not rested any part of it on tokens.

The result-digest comparison shows the fold is correct for `SUM`, which is the easiest possible operator. That recursive doubling and `reduce_bcast` agree on a content-addressed result at eight sizes is real evidence that the remainder path delivers and folds every contribution — if the uncounted pre-phase message had been lost, the sum would have been short. But every rank contributes the literal integer 1, so the sum is p and any fold that touches every contribution once produces it. The check confirms completeness, not order-sensitivity, and it would not detect an operator applied with its operands swapped. For the same reason, the observation that recursive doubling's sends reduce to only five distinct digests at $p = 32$ — one per round — says nothing about inter-rank agreement: with every contribution equal, every rank in a round holds the same partial sum by arithmetic.

The round-count discrepancy is inferred, not measured. I counted distinct tag phases per rank and treated that as the number of dependent communication steps. That is sound for this algorithm, because each phase's receive strictly precedes the next phase's send within a rank, but it is a reading of the tag structure plus the source, not a timing measurement. A run with a per-round cost large enough to see — any run with a real executor — would settle it directly, and none exists in this family.

Two campaigns are not two independent implementations. `mb-free` and `mb-smoke` agree exactly at all eight sizes measured twice, which rules out execution-order effects and per-run flakiness. It does not rule out anything systematic: both campaigns ran the same code at the same commit, so their agreement is a reproducibility check and not a second opinion. The second opinion in this analysis is the `msg.send` stream, which is produced by the transport rather than by the algorithm's own bookkeeping, and that is the only reason the undercount was visible at all.

The generated caption says "21 process counts" and means 21 runs. The family holds 21 rows at 13 distinct process counts — `mb-free` at all 13, `mb-smoke` repeating 8 of them — so "9 rows agree, 12 fall short" is a row count, not a count of p. In terms of p: 5 of the 13 are powers of two and agree, and all 8 that are not fall short by rem. Neither reading changes the conclusion, but the caption wording is worth correcting in `scripts/analyze_family.py`, since every family document inherits it.